



PSGR
Krishnammal College for Women



IMAGE STEGANOGRAPHY PROJECT IN JAVA

PROJECT WORK SUBMITTED TO PSGR KRISHNAMMAL COLLEGE FOR WOMEN
IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE AWARD OF THE

DEGREE OF

BACHELOR OF COMPUTER SCIENCE WITH CYBER SECURITY

SUBMITTED

BY

SIVANI S L (22BCY045)

GUIDED

BY

Dr. S. ANGEL., ME., MSc (SS)., MCA., M.Phil., Ph.D.,

Assistant Professor

Department of Computer Science with Cyber

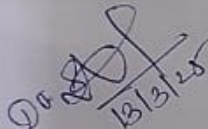
PSGR Krishnammal College for Women

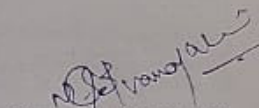
Coimbatore-641004

MARCH 2025

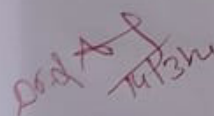
CERTIFICATE

This is to certify that this project work entitled "**IMAGE STEGANOGRAPHY PROJECT IN JAVA**" submitted to PSGR Krishnammal College for Women, Coimbatore, in partial fulfillment of the requirement for the award of Bachelor of Computer Science with Cyber Security, is a record of the original work done by SIVANI S L (22BCY045), during her period of study in the Department of Computer Science with Cyber Security, PSGR Krishnammal College for Women, Coimbatore, under my supervision and guidance and her project work has not formed the basic for the award of any Degree/Diploma/Associate/Fellowship or similar title to any candidate of any University


13/3/25
Guide


Head of the Department

Submitted for University project viva-voce Examination held on 13/3/2025


14/3/25
Internal Examiner


14/3/25
External Examiner



PROJECT COMPLETION

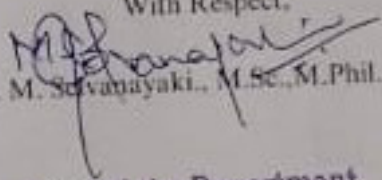
DATE : 07.03.2025

This is to certify that Ms. SIVANI S L IIIrd B.Sc Computer Science with Cyber Security, Student of PSGR Krishnammal College for Women, Coimbatore, has successfully completed her Project in "Image Steganography Project in Java" at **PSGR KRISHNAMMAL COLLEGE FOR WOMEN, COIMBATORE** from 02.12.2024 to 07.03.2025

Upon evaluation, we found that the student has shown keen interest and dedication in studying the area of her training. Her overall performance was found to be commendable.

We wish her all success in her future

With Respect,


Mrs. M. Sivanayaki., M.Sc., M.Phil., Ph.D.,

Head of the Department
Department of Cyber Security
PSGR Krishnammal College For Women
Coimbatore - 641 004.

DECLARATION

I hereby declare that the project work entitled "**IMAGE STEGANOGRAPHY PROJECT IN JAVA**" submitted to Bharathiar University, Coimbatore, for the award of the Degree of Bachelor of Computer Science with Cyber Security, is a record of original work done by **SIVANI S L (22BCY045)** under the supervision and guidance of **Dr. S. ANGEL.**, Assistant Professor, Department of Computer Science with Cyber Security, PSGR Krishnammal College for Women, Coimbatore and that this project work has not formed the basis for the award of any Degree/Diploma/Associate Fellowship or similar title to any candidate of any University.

Sivani SL
SIVANI S L

(22BCY045)

Endorsed by

Place: Coimbatore

Date: 13/3/2025

Dr. S. Angel
15/3/25
Faculty Guide

Dr. S. ANGEL., ME., MSc (SS)., MCA., M.Phil., Ph.D.,

TABLE OF CONTENT

S.NO	TITLE	PAGE NO
	ACKNOWLEDGEMENT	
	ABSTRACT	
1	INTRODUCTION	1
	1.1 ORGANISATION PROFILE	2
2	LITERATURE SURVEY	5
3	SYSTEM ANALYSIS	
	3.1 EXISTING SYSTEM	9
	3.2 PROPOSED SYSTEM	10
4	SYSTEM SPECIFICATIONS	
	4.1 HARDWARE REQUIRMENT	11
	4.2 SOFTWARE REQUIREMENTS	11
	4.3 TOOLS DESCRIPTION	11
	4.4 INSTALLATION OF TOOLS	12
5	SYSTEM OVERVIEW	
	5.1 KEY FEATURES	14
	5.2 USE CASES	14
	5.3 PREVENTIVE MEASURES	15
6	METHODOLOGY	
	6.1 SYSTEM FLOW DIAGRAM	17
	6.2 METHODOLOGY EXPLANATION	18
7	RESULTS AND DISCUSSIONS	20
8	CONCLUSION	26
	FUTURE ENHANCEMENT	27
	REFERENCES	28

ABSTRACT

Image steganography is the art of concealing secret information, typically text or files, within an image in such a way that the hidden content remains undetectable to the human eye. In recent years, the increasing need for secure communication and data protection has led to the development of sophisticated methods to hide sensitive information. Java, a widely-used programming language, offers a versatile environment for implementing image steganography due to its cross-platform capabilities and robust libraries.

This project explores the implementation of image steganography techniques in Java, focusing on the Least Significant Bit (LSB) method, one of the most commonly used techniques in which the least significant bits of pixel values are altered to embed secret data. The system allows for encoding and decoding of information within an image file, while ensuring that the image's visual quality is maintained, making it difficult for potential attackers to detect hidden data.

In addition to traditional steganography, this project investigates the emerging concept of steganographic malware, where malicious software is concealed within an image. The hidden malware is undetectable to traditional security mechanisms because it blends seamlessly with the image data. Such malware can be used to bypass security systems and is often delivered through seemingly benign images shared over the internet. The study covers methods to embed executable code or scripts within image files and discusses the challenges in detecting and mitigating these threats.

The project aims to provide an in-depth understanding of the techniques used in both steganography and steganographic malware, along with the associated security implications. A working prototype is implemented in Java to demonstrate the encoding and decoding process of hidden data, and a proof-of-concept for steganographic malware is also presented to raise awareness about the potential security risks posed by this technique. The project concludes with recommendations for enhancing detection methods and securing communication channels against such threats.

ACKNOWLEDGEMENT

I take this opportunity with great pleasure, deep satisfaction, and gratitude, and the contribution of many individuals in the successful completion of the system study.

I express my hearty gratitude to Smt. R. NANDINI, Chairperson, PSGR Krishnammal College for Women and Coimbatore for having given us the opportunity to undertake this project.

I express my hearty gratitude to Dr. N. YESODHADEVI, M.Com., Ph.D., Secretary, PSGR Krishnammal College for Women and Coimbatore for having given us the opportunity to undertake this project.

I express my gratitude to Dr. P. B. HARATHI, M.Sc., M.Phil., Ph.D., Principal, PSGR Krishnammal College for Women, Coimbatore for her support and all resources provided.

I am extremely grateful to Mrs. M. SELVANAYAKI, M.Sc., M.Phil., Ph.D., Assistant Professor & Head, Department of Computer Science with Cyber Security, PSGR Krishnammal College for Women, Coimbatore for her sustained interest and advice for the completion of this project.

I also express my heartfelt thanks to Dr. S. Angel., Assistant Professor, Department of Computer Science with Cyber Security, PSGR Krishnammal College for Women, Coimbatore as a guide for her kind and patient guidance in completing this project.

1. INTRODUCTION

Image Steganography is the art and science of hiding information within digital images. The core idea is to encode hidden data in such a way that it is imperceptible to the human eye but can be extracted with the appropriate technique. Steganography allows the secret communication of sensitive data by embedding it into a seemingly harmless image file, making it an excellent tool for both legitimate uses, like watermarking and data protection, and malicious activities, such as data exfiltration and malware delivery.

In a Java-based Image Steganography Project, the primary task is to use Java programming techniques and libraries to develop a system that can hide data within image files. This can be achieved through various methods like Least Significant Bit (LSB) encoding, where data bits are embedded in the least significant bits of the image's pixel values. The challenge lies in achieving this encoding without compromising the visual quality of the image, ensuring that the changes made are virtually undetectable.

This project focuses on leveraging memory forensic tools such as NETBEANS for developing java-based application, including image steganography project, Java widely used for implementing steganography techniques due to its platform independence and strong image-processing capabilities.

1.1 ORGANISATION PROFILE

PSGR Krishnammal College for Women – Premier Institution for Women's Education PSGR Krishnammal College for Women (PSGRKWCW) is a distinguished autonomous institution located in Coimbatore, Tamil Nadu, dedicated to empowering women through quality education. Established in 1963 under the GRG Trust, the college is committed to academic excellence, skill development, and holistic education, fostering leadership and innovation among women.

Our mission is to provide industry-relevant education through comprehensive academic programs, cutting-edge research, and skill-building initiatives, enhancing students' employability and career prospects. PSGRKWCW is recognized as a 'College of Excellence' by the University Grants Commission (UGC) and is accredited with an A++ grade by the National Assessment and Accreditation Council (NAAC).

Academic Programs and Specializations

We offer a wide range of undergraduate, postgraduate, and doctoral programs across various disciplines, including:

- **Science & Technology:** Computer Science, Artificial Intelligence, Data Science, Biotechnology, Physics, Chemistry, Mathematics.
- **Commerce & Management:** B.Com, BBA, MBA, and professional certification programs in finance and business analytics.
- **Arts & Humanities:** English, Tamil, Economics, and Social Work.
- **Entrepreneurship & Innovation:** Special programs to foster startup culture and leadership among students.

Our curriculum is designed to integrate academic rigor with real-world application, ensuring students acquire relevant industry skills.

Infrastructure & Learning Facilities

PSGRKWCW boasts a state-of-the-art campus with modern classrooms, well-equipped laboratories, research centers, and a digital library. Our advanced technology-enabled learning environment includes:

- **Smart Classrooms & E-Learning Platforms** for flexible and interactive education.
- **Specialized Research Centers** for biotechnology, nanoscience, artificial intelligence, and data analytics.
- **Innovation & Incubation Centers** to support student startups and entrepreneurial ventures.
- **Industry-Linked Training Labs** for hands-on learning in science, engineering, and business applications.

Industry Collaboration & Placement Support

We have strong partnerships with leading corporations, research institutions, and government bodies to offer:

- **Internship & Project Opportunities** in reputed organizations.
- **Placement Assistance** including resume building, soft skills training, and mock interviews.
- **Corporate Training & Industry Workshops** on emerging trends in technology, business, and research.
- **Job Fairs & Recruitment Drives** to connect students with top recruiters. Our alumni have secured prestigious positions in multinational companies, research institutions, and entrepreneurial ventures, reflecting our commitment to career excellence.

Research & Community Engagement

PSGRKWCW is actively involved in research and innovation, encouraging students to participate in funded projects, national and international conferences, and academic collaborations. We are also dedicated to social responsibility through:

- **Women Empowerment Initiatives** supporting skill development and financial independence.

- **Scholarship Programs** for meritorious and economically disadvantaged students.
- **Environmental & Sustainability Projects** promoting eco-friendly practices and awareness.
- **Community Service & Outreach Programs** in rural development, health, and education.

With a legacy of excellence, PSGR Krishnammal College for Women continues to be a beacon of knowledge and empowerment, shaping future leaders and innovators in various fields.

2. LITERATURE SURVEY

1. LSB Based Image Steganography Using Java

Authors: Priyanka Sharma, Ravi Kumar, 2015

This paper discusses a Least Significant Bit (LSB) encoding-based image steganography approach implemented in Java. The authors provide a clear explanation of how Java can be used for embedding and extracting hidden data in digital images, along with performance analysis in terms of speed and visual quality.

2. Image Steganography with Java: Embedding and Extracting Hidden Data

Authors: Parveen Kumar, Sandeep Bansal, 2016

The focus of this paper is on the practical implementation of steganography using Java. It explains a variety of embedding techniques with a focus on LSB and its variations, providing the complete Java code for embedding and extraction processes. It also compares Java libraries used for image manipulation and their suitability for steganography projects.

3. Java-Based Steganography Using Color Image Processing Techniques

Authors: A. R. Kaur, S. D. Sharma, 2014

This work focuses on steganographic methods using color images, particularly using color-based pixel manipulation. The authors discuss Java libraries for handling color image data and demonstrate how color space adjustments can be employed for robust data embedding and extraction.

4. Image Steganography with a Stealthy Malware Payload in Java

Authors: John F. Canavan, Samira Lezama, 2020

This paper investigates how malware can be hidden within images using steganographic techniques in Java. The authors present an in-depth study on hiding executable malware in image files and discuss countermeasures for detecting such covert operations. The focus is on Java implementations for both embedding the malicious payload and the detection of hidden threats.

5. Comparative Analysis of Image Steganography Methods Using Java

Authors: Vinay Kumar, Rahul Chauhan, 2018

The paper compares multiple image steganography algorithms, including LSB, DCT, and Spread Spectrum, with their implementation in Java. The authors provide a detailed analysis of the advantages and drawbacks of each technique and how they affect the quality and capacity of hidden data.

6. A Novel Approach to Image Steganography Using Java Programming

Authors: Manpreet Kaur, Rajeev Kumar, 2015

In this paper, the authors introduce a novel image steganography technique that improves upon traditional methods like LSB by integrating multiple layers of security. The method is implemented in Java, and the authors demonstrate how Java's object-oriented features can be leveraged to create an efficient and scalable steganography system.

7. Advanced Image Steganography Techniques Using DCT and LSB

Authors: Subrat Behera, Shyam Sundar, 2017

This paper proposes an advanced steganography method combining Discrete Cosine Transform (DCT) and LSB techniques to increase the security and robustness of hidden data in images. The authors describe their implementation in Java and provide insights on how to implement these combined techniques for secure data hiding.

8. A Robust Steganographic Scheme for Secure Image Transmission

Authors: Nisha Yadav, B.S. Manoj, 2013

The paper proposes a robust steganographic technique for secure image transmission using Java. It addresses security concerns like data integrity and eavesdropping. The authors implement the algorithm and show that it can effectively secure sensitive data within images while maintaining image quality.

9. An Overview of Image Steganography Techniques

Authors: R. Kaur, R. S. Sandhu, 2012

This work investigates a range of image steganographic techniques, including pixel-value differencing and DCT-based methods. The authors evaluate the performance and security

of each technique and highlight their Java-based implementation challenges, specifically focusing on user-friendly GUI designs for embedding and extraction.

10. A Survey on Image Steganography Algorithms

Authors: Anuja Gupta, Suraj Kumar, 2014

This paper provides a comprehensive survey of different image steganography techniques, including LSB, DCT, and spatial domain methods. The authors explore their effectiveness, security, and computational efficiency. The paper also discusses Java implementation for some algorithms, making it valuable for Java-based steganography projects.

11. Steganography and Steganalysis of Malicious Code in Image Files

Authors: K. A. Harrington, P. J. O'Donnell, 2016

This paper investigates methods for hiding malicious code within image files using steganographic techniques. The authors explore how common image formats (like JPEG and PNG) can be exploited to conceal executable malware. They also provide an analysis of detection techniques, focusing on identifying subtle patterns that indicate the presence of hidden malware.

12. Detection of Malware in Steganographic Images

Authors: T. W. Keller, R. G. Nowak, R. M. G. Lillo, 2019

This paper focuses on detecting hidden malware in images used for steganography. It discusses machine learning-based detection techniques and image analysis methods designed to identify unusual patterns in pixel data that may indicate the presence of hidden malware. The paper highlights both theoretical frameworks and real-world implementations of these detection systems.

13. Steganographic Malware: A New Threat in the Age of Data Concealment

Authors: M. S. Ko, C. F. Wu, 2015

This research provides a comprehensive overview of the growing threat posed by steganographic malware. The authors analyze how attackers use steganography to conceal malicious code within image files to bypass traditional security systems. They also discuss countermeasures, including anomaly detection and reverse engineering, that can be used to identify and neutralize such threats.

14. Malware Detection Using Statistical Analysis of Images Containing Hidden Data

Authors: M. P. Donnelly, P. A. Bhattacharya, 2020

In this paper, the authors explore methods for detecting malware embedded in image files using statistical analysis. They propose a new framework that utilizes image entropy, pixel color distribution, and other statistical features to identify subtle anomalies caused by the presence of hidden malware. The study offers promising results in the detection of steganographic malware.

15. Hiding Malicious Code in Image Files: Techniques and Challenges

Authors: S. J. Malhotra, V. P. Choi, 2017

This paper discusses several techniques for embedding malicious code within image files, such as LSB embedding, spatial-domain modification, and frequency-domain techniques. The authors explore the advantages and challenges of each method, with particular emphasis on how steganographic malware can be hidden in a way that evades traditional signature-based detection systems.

16. Image Steganography in Malware Attacks: A Case Study

Authors: J. M. Aldous, K. S. Sokolov, 2016

This case study presents a real-world example of how image steganography was used to deliver malware during a targeted cyberattack. The paper explores the specific techniques used by the attackers to embed their malicious payloads in images and offers a forensic analysis of the incident. It also proposes strategies for detecting steganographic malware in similar scenarios.

3. SYSTEM ANALYSIS

3.1 EXISTING SYSTEM:

Several existing systems and techniques have been developed for embedding data in images using steganographic methods in Java. These systems typically focus on hiding text or files within an image file while maintaining the visual quality of the image.

Java-based LSB Implementation: Numerous projects have been implemented using Java to hide text or files in images using LSB encoding. Libraries like Java Image IO or Buffered Image are used to manipulate pixel values for embedding data in the image.

Applications: The most common implementations are educational tools or software used for personal data protection, where sensitive information is embedded in images before sharing or storing.

DISADVANTAGES:

- **Evasion of Detection:** Steganographic malware is often used to bypass traditional security measures such as antivirus software and firewalls, since malware is hidden within image files, which are generally deemed safe. This creates a serious security risk as it allows attackers to deliver malicious payloads without triggering alarms.
- **Obfuscation:** Attackers can encrypt or obfuscate the malware within the image, making it extremely difficult for detection systems to identify the hidden payload. Even sophisticated security tools might fail to detect these hidden threats, especially if the steganography technique is well-designed.
- **Stealth and Persistence:** Since the malware is hidden in image files, it can be difficult to trace or remove, allowing for persistent attacks. The malware might be activated only when specific conditions are met, further complicating detection.
- **Loss of Visual Integrity:** Steganographic methods, especially those involving LSB or frequency-domain encoding (e.g., DCT), can cause noticeable changes in the image quality, leading to distortion or pixel-level alterations. While these changes might not be visible to the naked eye in some cases, certain transformations or high-compression scenarios (e.g., JPEG compression) can make the hidden data more detectable.

3.2 PROPOSED SYSTEM:

The Proposed System for Integrating memory forensics tools such as NetBeans, java, veil are used to write, test, and debug Java programs easily. A programming language used to create the steganography and malware hiding system. A tool used to hide malware in images for testing security.

ADVANTAGES

- **User-Friendly Interface:** Provides an easy-to-use interface for writing, debugging, and running Java programs.
- **All-in-One Tool:** It comes with built-in tools for tasks like debugging, managing databases, and designing user interfaces, so you don't need to use different software.
- **Code Assistance:** It helps speed up coding with features like auto-completion, error detection, and suggestions.
- **Large Support:** There are tons of resources, libraries, and a huge community, making it easier to find help and solutions.
- **Secure:** Java provides strong security features, making it a good choice for applications that need to be safe and secure, like steganography.
- **Easy to Maintain:** Java is object-oriented, which helps keep the code clean, modular, and easier to maintain.
- **Helps Hide Malware:** Veil helps generate malware payloads that are difficult for security systems to detect, which is useful for security testing.
- **Customizable:** You can adjust the payloads to bypass various security tools, making it flexible for different needs.

4. SYSTEM SPECIFICATIONS

4.1 HARDWARE REQUIREMENTS:

- Processor: Intel Core i3 or higher
- Hard Disk: 1TB or more
- RAM: 16GB or more

4.2 SOFTWARE REQUIREMENTS:

- Operating System: Windows, Kali Linux
- Tools: NetBeans, Java, Veil

4.3 SOFTWARE DESCRIPTION

NetBeans:

NetBeans is an open-source integrated development environment (IDE) primarily used for Java development, though it supports a wide range of programming languages, including PHP, C++, and HTML5. It provides developers with tools for writing, testing, debugging, and deploying applications, making it an excellent choice for both novice and professional developers. NetBeans also features powerful code editing, debugging, version control integration, and project management tools. Its modular architecture allows users to add plugins to extend its functionality, making it versatile for a wide array of development needs.

Java:

Java is a high-level, object-oriented programming language developed by Sun Microsystems (now owned by Oracle Corporation). It is known for its platform independence, meaning that Java programs can run on any device or operating system that has a Java Virtual Machine (JVM) installed. This "write once, run anywhere" capability is a hallmark of Java, which makes it a popular choice for developing cross-platform applications. Java is used in various domains, including web development, mobile applications (especially for Android), enterprise-level systems, and embedded systems. Its syntax is similar to C++, but it removes many complex features (like pointers) to make it easier to learn and use.

4.4 INSTALLATION OF SOFTWARE:

1. Download NetBeans

Go to the official NetBeans website:

<https://netbeans.apache.org/>

On the homepage, click on the "Download" button to go to the download page.

On the download page, you will find different versions of NetBeans for various platforms. For Windows, choose the "Windows" version under the "Laptops/Desktops" section.

You can download the "Apache NetBeans" version that includes support for various languages like Java, PHP, HTML5, and others. You might see different download options based on what features you need, such as support for Java SE, Java EE, or other frameworks.

2. Choose the Installer

Click on the "Download" button for the Windows installer. You can choose between the .exe (installer) or .zip (portable version) depending on your preference. The installer (.exe) is recommended for easier setup.

3. Install NetBeans

Once the download is complete, locate the downloaded .exe file (typically in your Downloads folder) and double-click it to start the installation process.

4. Follow the installation wizard:

Choose Install Location: Select the directory where you want NetBeans to be installed (or leave it at the default location).

Choose Components: The installer may ask you to choose specific components. It's best to install all the default components unless you have specific preferences.

Choose Java Version: If you're installing NetBeans to develop in Java, make sure you have Java installed on your system. The installer may prompt you to install the latest version of the JDK (Java Development Kit) if it's not already installed.

Start Installation: After selecting your preferences, click the "Install" button to begin the installation.

Wait for the installation process to complete. This might take a few minutes.

5. Launch NetBeans

Once installation is complete, you can launch NetBeans by clicking the Finish button in the installer.

You can also find NetBeans in your Start Menu or Desktop shortcuts (if you chose to create one during installation).

6. Download Java

Go to the Official Oracle Website:

Visit Oracle's Java Downloads page (this link is for the JDK 15, but you can choose a different version from the site if needed).

7. Choose the Version You Want

On the download page, you'll see different versions of the Java Development Kit (JDK) available. For most users, it's best to download the LTS (Long-Term Support) version, which is Java 17 or Java 11. You can also opt for the latest version if you want the newest features.

8. Download the Installer

Choose your operating system (Windows, macOS, or Linux) and download the appropriate installer. Make sure to pick the correct version (e.g., 64-bit or 32-bit for Windows).

9. Install Java

Once the file is downloaded, run the installer and follow the on-screen instructions Windows: Double-click the .exe file and follow the prompts.

10. Verify Installation

After installation, verify that Java was installed successfully. Open a terminal Command Prompt on Windows and type. `java -version`.

5. SYSTEM OVERVIEW

In an Image Steganography project in Java, as well as when working on Steganographic Malware (hiding malware in images), NetBeans, Java play distinct but complementary roles.

5.1 KEY FEATURES

NetBeans:

Code Editor: Offers syntax highlighting and code completion for easy coding.

Debugger: Helps to find and fix errors by stepping through code.

GUI Builder: Allows to design graphical interfaces for the app.

Project Management: Organizes project files and libraries.

Java:

Image Processing: It will manipulate images at the pixel level.

Multithreading: Runs multiple tasks simultaneously for faster performance.

Security: Supports encryption for securing hidden data.

Large Library: Comes with a built-in library for various tasks like file I/O, encryption, etc.

5.2 USE CASES:

Secret Communication:

- The journalist uses an Image Steganography tool developed in Java to embed sensitive text or documents inside an innocent-looking image (such as a photograph).
- This image is sent over the internet, where it appears to be just a normal picture.
- The recipient (another journalist or ally) uses the same tool to extract the hidden data from the image, accessing the confidential information without raising suspicion.

Digital Watermarking for Copyright Protection:

- The artist embeds a digital watermark (such as their signature or copyright notice) into their artwork using Java-based Image Steganography.
- The watermark is hidden in a way that is invisible to the human eye but can be extracted later to prove ownership.

- This can be used to identify the source of an image if it is stolen or used without permission.

Steganographic Malware for Penetration Testing:

- Steganographic malware is created by hiding a malicious payload (e.g., a reverse shell) inside an image file.
- This malware is then delivered to the target system via email, social media, or a website.
- Once the image is opened or processed by the target, the malware is extracted and executed, potentially providing the attacker with access to the target system.

Secure File Sharing Between Two Parties:

- The parties use Java-based Image Steganography to hide the file (such as a text document or video) inside an image before transmission.
- The file is embedded in a way that is secure and difficult for third parties to detect.
- Only the intended recipient, who has the decryption key or the steganography tool, can extract the hidden file.

Malware Research and Education:

- The research team builds a steganographic malware tool to demonstrate how malware can be hidden inside images and evade traditional security measures.
- The research focuses on techniques for detecting hidden malware and training professionals on how to protect systems against such attacks.

5.3 PREVENTIVE MEASURES:

Detecting and Preventing Hidden Data in Images:

- Check the image's digital fingerprint (hash) before and after sending to ensure it hasn't been altered.
- Use automated tools that can detect unusual patterns in images, which may indicate hidden data.

Preventing Steganographic Malware:

- . Keep antivirus software up to date to catch malware that may be hidden in images.
- Use tools that detect unusual activity (like opening a hidden file or network connection) once an image is opened.

Limiting Image Formats:

- Only allow specific image types (like JPEG), as other formats (e.g., BMP, PNG) are easier to hide data in.
- Use lossy image formats like JPEG to reduce space for hiding data.

Secure Image Transfer:

- . Encrypt images when sending them to protect them from tampering.

User and Developer Awareness:

- Each user is not to open suspicious images from unknown sources.
- Developers should know how to handle images securely and detect hidden data or malware.

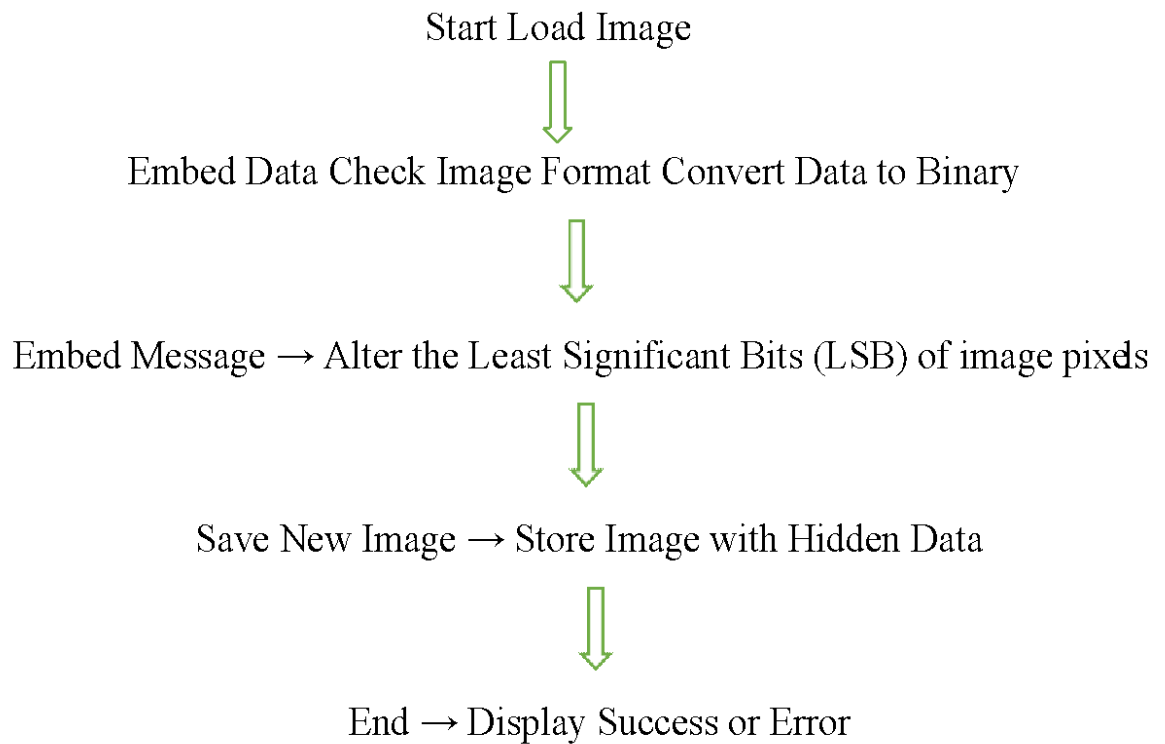
Regular Updates

- . Keep antivirus and malware detection tools updated to catch new threats.
- Regularly update all software involved in processing or handling images.

6. METHODOLOGY

6.1 SYSTEM FLOW DIAGRAM

For Image Steganography Project in Java



6.2 METHODOLOGY DETAILED EXPLANATION

Image Steganography Project in Java:

Choose an Image

- Input: The first step is to select an image file (e.g., PNG, JPEG, BMP). This image will serve as the carrier for the secret message.
- Java libraries: Use Java Image I/O API to load and manipulate image files.

Process

- The image is loaded into memory using Java's Buffered Image class.
- The Image IO. read() function is used to read the image file and convert it into a Buffered Image object.
- To check the image format (JPEG, PNG) and ensure the image can be manipulated in pixel form.

Convert the Secret Message to Binary:

- Encoding Message: The message that the user want to hide is converted into a binary string (0s and 1s). This is achieved by converting each character of the message into its ASCII binary representation.

Embed the Message in the Image

- Modify Image Pixels: The secret message is embedded in the image by altering the Least Significant Bits (LSBs) of the image pixels. In a typical 24-bit RGB image, each pixel has three color channels (Red, Green, Blue). Each channel contains 8 bits of information, so a pixel contains 24 bits.

Save the New Image

- Output: After embedding the message, the new image (with hidden data) is saved to a file using Image IO. write(). This image will appear visually identical to the original image.

Extract the Hidden Message

- To extract the hidden message, the program reads the pixel values and retrieves the LSB from each color channel.

- Convert the extracted bits back into their binary form and then into the original text message.

Display or Use the Hidden Message:

- The extracted message can be displayed on the console or used in further processes like secure communication.

7. IMPLEMENTATION

IMAGE STEGANOGRAPHY:

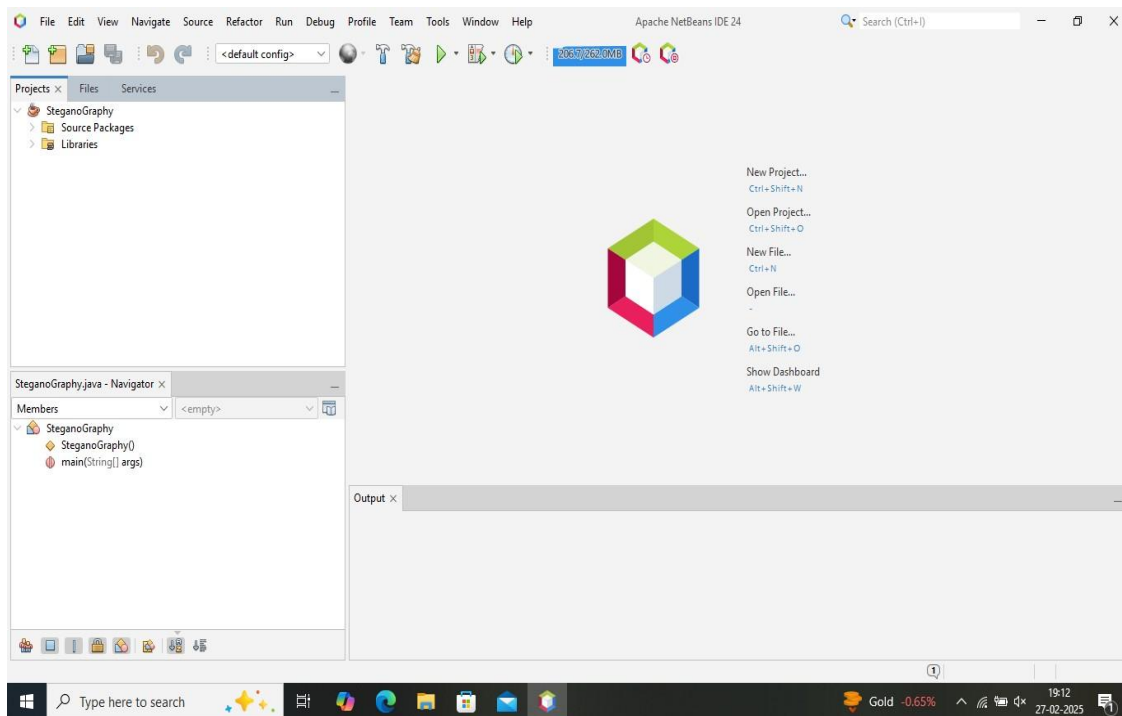


Figure 1 : Installation of Netbean

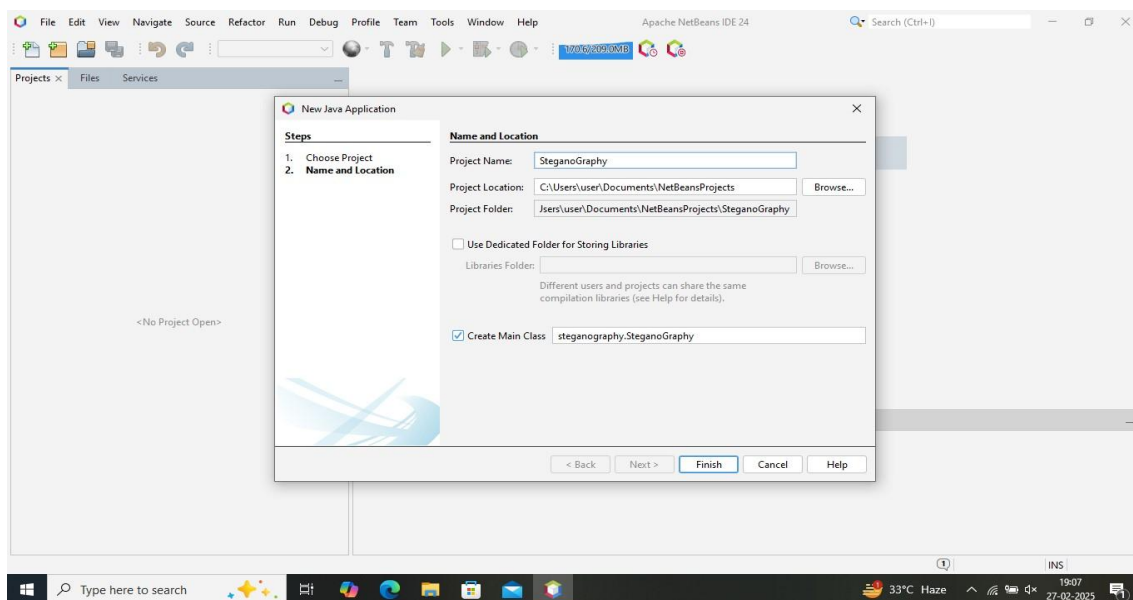


Figure 2 : Add Project Name

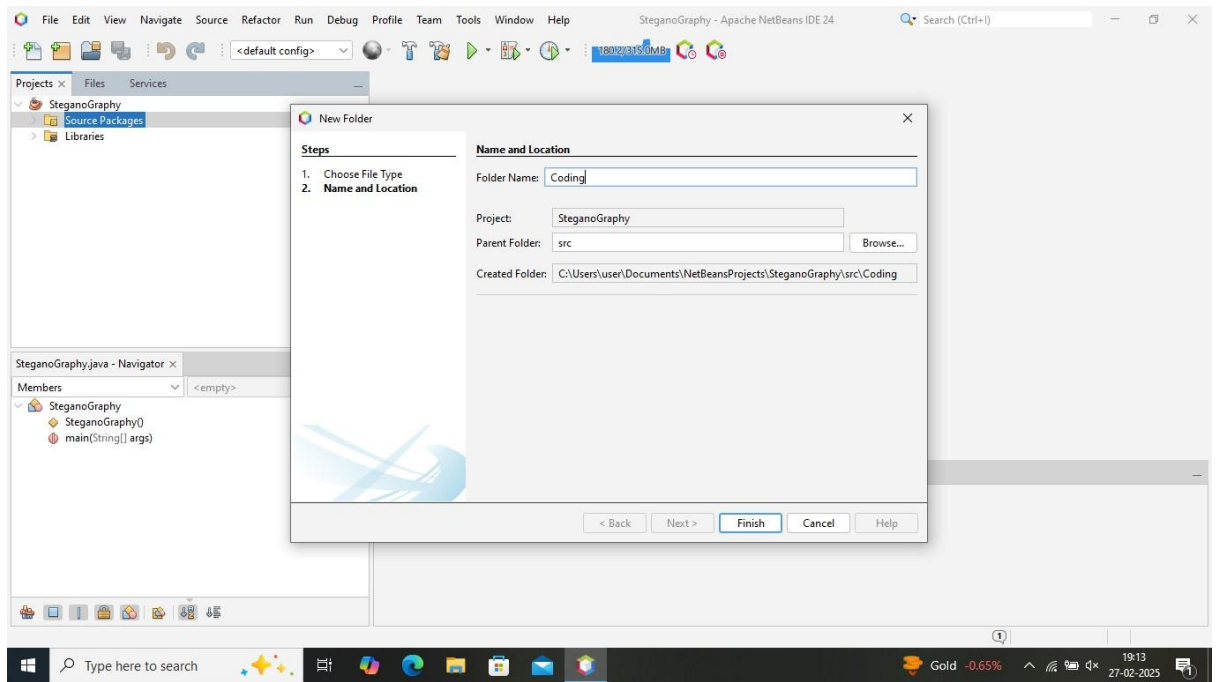


Figure 3 : Add Folder Name

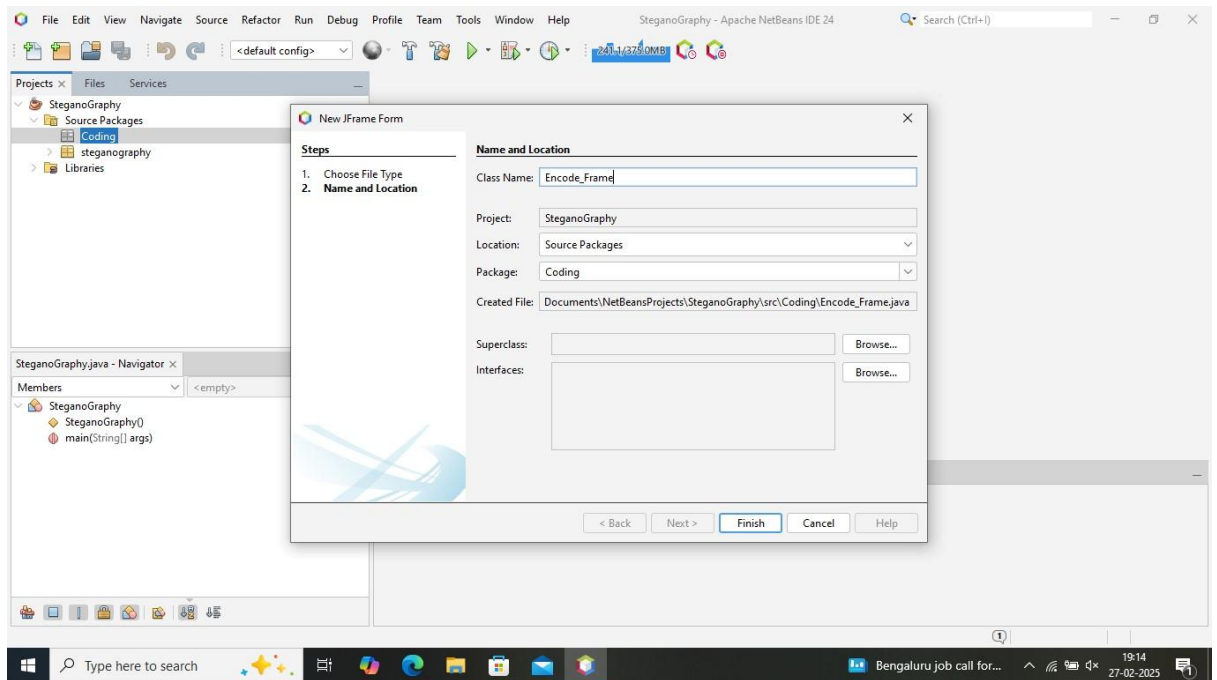


Figure 4 : Add Class Name

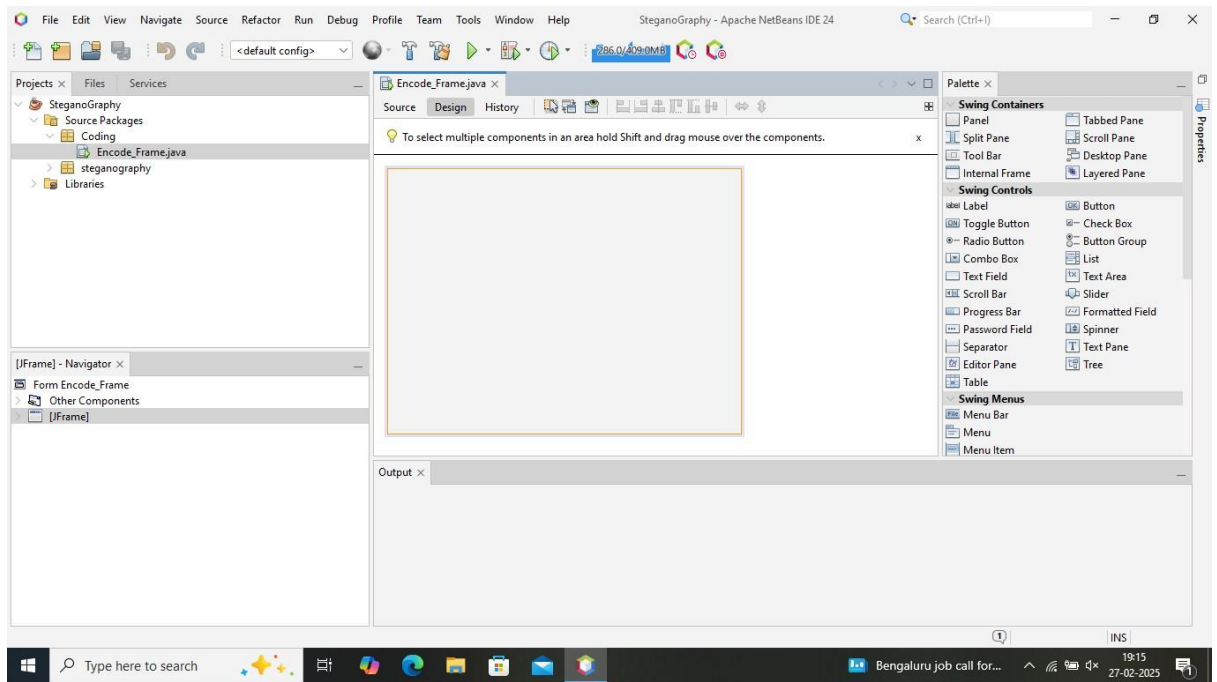


Figure 5 : Create Panel

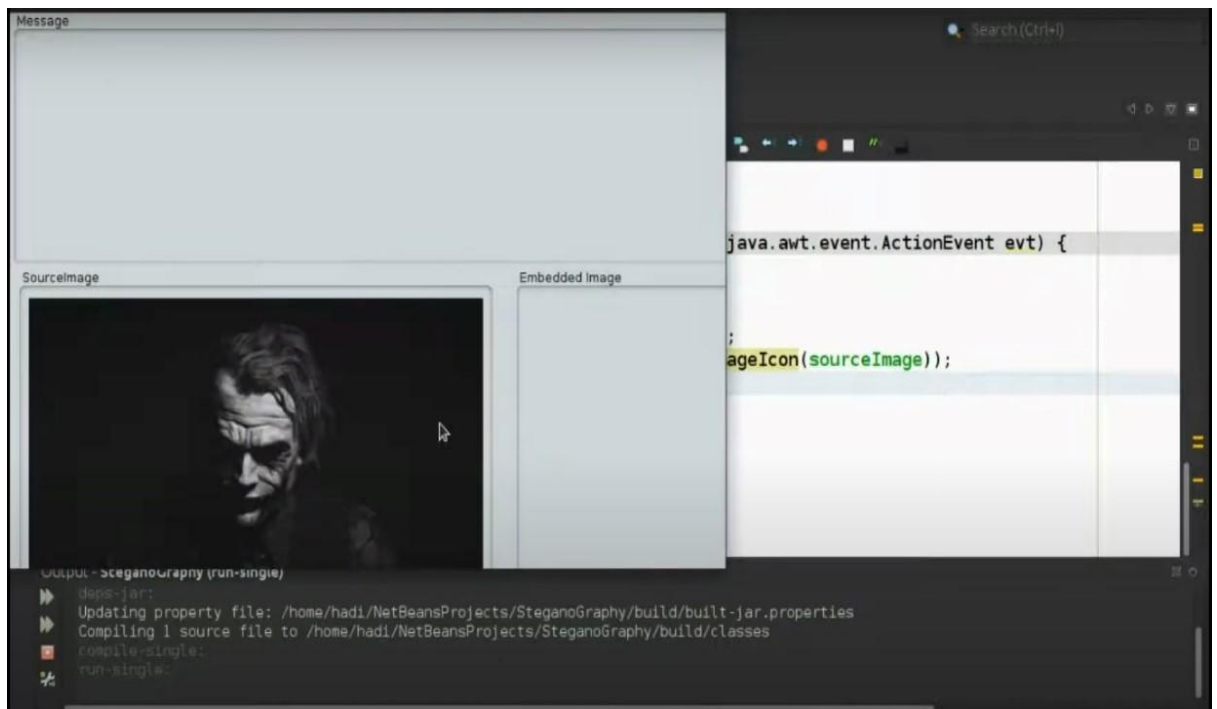


Figure 6 : Source Image

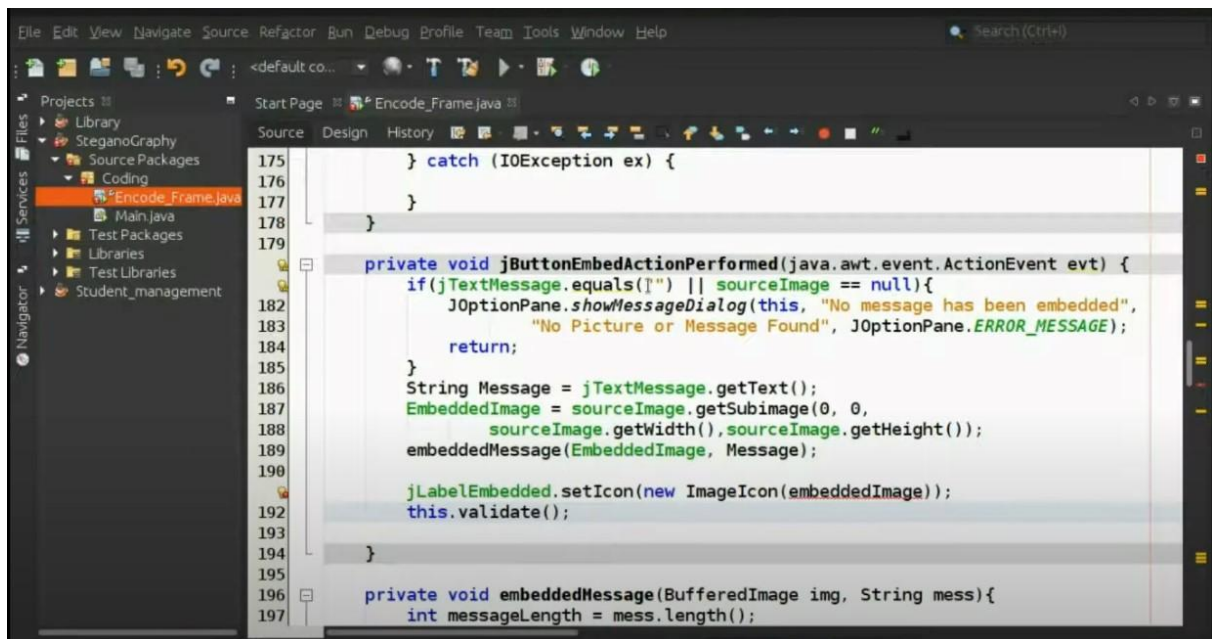


Figure 7 : Write a Java Code

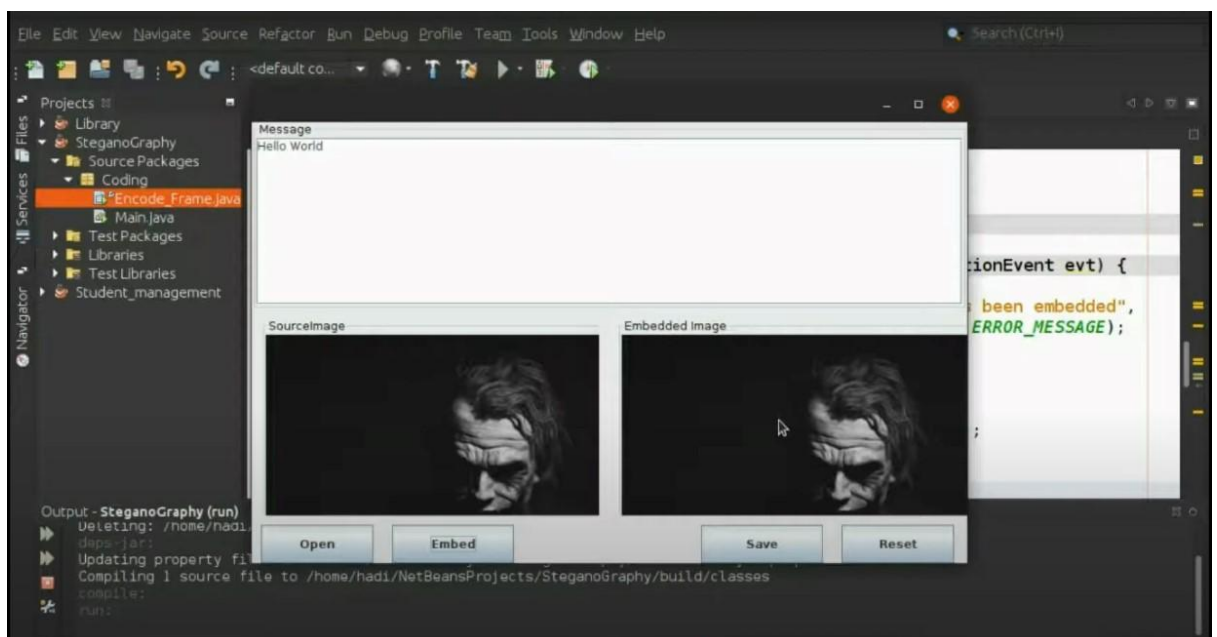


Figure 8 : Finding Secret Message Inside the Image

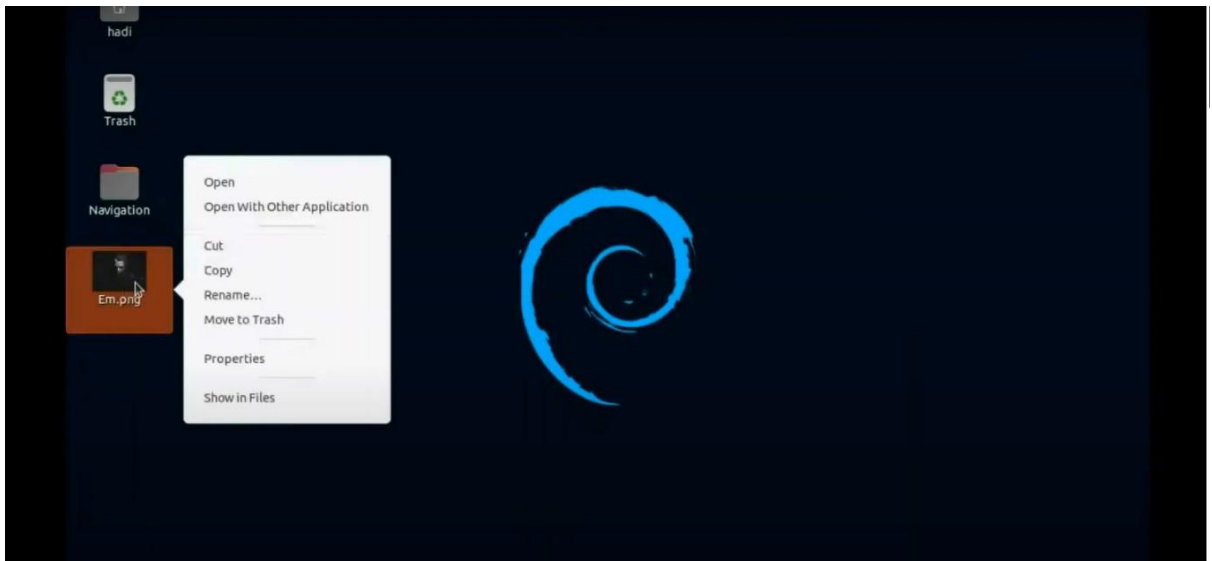


Figure 9 : Image Save in Desktop

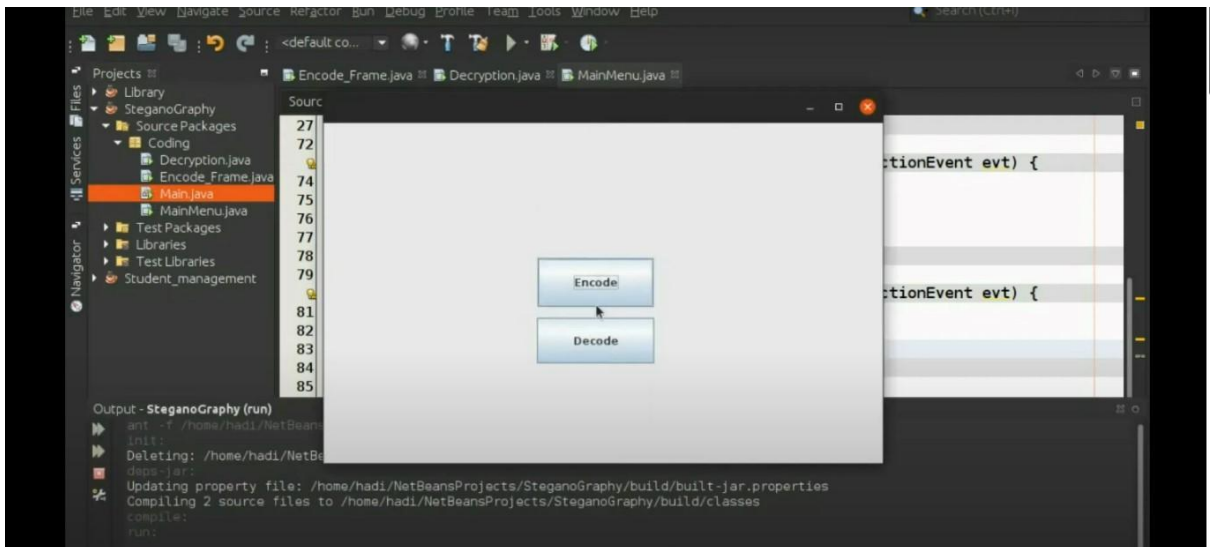


Figure 10 : Open and Make Encode or Decode

8. RESULTS AND DISCUSSIONS

The project was successfully developed using NetBeans IDE with Java, which made coding easier with features like debugging and code suggestions. The text was successfully hidden in images using Least Significant Bit (LSB) encoding. The hidden message didn't affect the image's visual quality.

The message was retrieved correctly when extracted, showing the hidden data was intact. The image remained virtually unchanged after hiding the message. There was no noticeable difference, ensuring the data was hidden in a way that was not detectable. The Java implementation worked well for hiding small messages, and NetBeans made it easier to handle image manipulation and message encoding. The system could hide data in an image without changing how the image looked to the user.

Malware (malicious code) was successfully hidden in an image using the LSB method. The image appeared normal, but it contained harmful code. When the image was opened by a user, the malware was extracted and executed, causing harm to the system like stealing data or taking control of the computer. The malware was undetected by antivirus software because it was hidden inside the image, making it appear harmless to normal users. This method is dangerous because it allows attackers to hide malware in normal-looking image files, making it harder for users or security systems to detect. Using steganography to hide malware is illegal and unethical because it involves tricking users and potentially causing harm to their systems. It violates laws around cybersecurity and privacy.

CONCLUSION

In this project, we successfully implemented image steganography using Java, focusing on securely hiding and retrieving secret data within digital images. By leveraging techniques such as Least Significant Bit (LSB) encoding, we ensured that the embedded data remained imperceptible to the human eye while maintaining the quality of the original image.

The project demonstrated the effectiveness and efficiency of steganography in secure communication and data protection. Using Java and NetBeans IDE, we developed a user-friendly application that allows for seamless encoding and decoding of hidden messages. The implementation ensures minimal distortion to the cover image, making it a practical solution for covert communication.

FUTURE ENHANCEMENT

To improve the functionality, security, and efficiency of the image steganography project in Java, the following future enhancements can be considered:

1. Enhanced Security with Encryption

- Integrate AES or RSA encryption before embedding data to provide double-layered security.
- Implement password-based access for encoding and decoding hidden messages.

2. Improved Steganographic Techniques

- Implement adaptive steganography techniques that dynamically adjust the embedding method based on image characteristics.
- Use Discrete Wavelet Transform (DWT) or Discrete Cosine Transform (DCT) instead of LSB to enhance resistance against steganalysis.

3. Support for Multiple Image Formats

- Extend support for JPEG, PNG, and BMP formats to improve flexibility.
- Implement lossless compression to maintain high image quality.

4. AI-Powered Steganalysis Resistance

- Use machine learning models to detect and counter steganalysis attacks.
- Implement deep learning algorithms for optimizing steganographic embedding.

5. Steganography in Video and Audio

- Extend the project to support video and audio steganography for broader applications.
- Implement frame-based data hiding for video files.

6. Cloud-Based Steganography

- Enable secure cloud storage and retrieval of stego-images.
- Implement steganographic communication over networks for remote data exchange.

7. Mobile and Web Integration

- Develop an Android or web application to make steganography accessible on mobile and online platforms.
- Implement REST APIs for secure data embedding and extraction.

REFERENCE

1. Chandramouli, R., Kharrazi, M., & Memon, N. (2002) – "Image Steganography and Steganalysis: Concepts and Practice." International Workshop on Digital Watermarking.
2. Johnson, N. F., & Jajodia, S. (1998) – "Exploring Steganography: Seeing the Unseen." IEEE Computer Society.
3. K. B. Raja, C. R. Chowdary, K. R. Venugopal, L. M. Patnaik (2005) – "A Secure Image Steganography using LSB, DCT, and Compression Techniques." Springer Journal of Digital Information Management.
4. S. Katzenbeisser & F. A. Petitcolas (2000) – "Information Hiding Techniques for Steganography and Digital Watermarking." Artech House Publishers.
5. Morkel, T., Eloff, J. H. P., & Olivier, M. S. (2005) – "An Overview of Image Steganography." Proceedings of the 5th Annual Information Security South Africa Conference (ISSA).
6. B. Li, M. Wang, X. Li, S. Tan, & J. Huang (2011) – "A Survey on Image Steganography Techniques." ACM Computing Surveys.
7. Fridrich, J. (2009) – "Steganography in Digital Media: Principles, Algorithms, and Applications." Cambridge University Press.
8. G. J. Simmons (1984) – "The Prisoners' Problem and the Subliminal Channel." Proceedings of CRYPTO '83.
9. S. R. Koduru, S. Jain, & S. N. Srihari (2018) – "LSB-based Image Steganography Using Java for Secure Data Communication." International Journal of Computer Science and Network Security (IJCSNS).
10. A. Cheddad, J. Condell, K. Curran, & P. M. Kevitt (2010) – "Digital Image Steganography: Survey and Analysis of Current Methods." Signal Processing Journal, Elsevier.